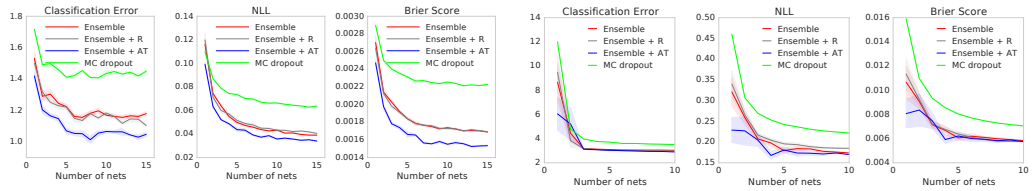


(a) MNIST dataset using 3-layer MLP (b) SVHN using VGG-style convnet

Figure 2: Evaluating predictive uncertainty as a function of ensemble size M (number of networks in the ensemble or the number of MC-dropout samples): Ensemble variants significantly outperform MC-dropout performance with the corresponding M in terms of all 3 metrics. Adversarial training improves results for MNIST for all M and SVHN when $M = 1$, but the effect drops as M increases.

To measure the sensitivity of the results to the choice of network architecture, we experimented with a two-layer MLP as well as a convolutional NN; we observed qualitatively similar results; see Appendix B.1 in the supplementary material for details.

We also report results on the SVHN dataset using an VGG-style convolutional NN.[8] The results are in Figure 2(b). Ensembles outperform MC dropout. Adversarial training helps slightly for $M = 1$, however the effect drops as the number of networks in the ensemble increases. If the classes are well-separated, adversarial training might not change the classification boundary significantly. It is not clear if this is the case here, further investigation is required.

Finally, we evaluate on the ImageNet (ILSVRC-2012) dataset [51] using the *inception* network [56]. Due to computational constraints, we only evaluate the effect of ensembles on this dataset. The results on ImageNet (single-crop evaluation) are shown in Table 4. We observe that as M increases, both the accuracy and the quality of predictive uncertainty improve significantly.

Another advantage of using an ensemble is that it enables us to easily identify training examples where the individual networks disagree or agree the most. This disagreement[9] provides another useful qualitative way to evaluate predictive uncertainty. Figures 10 and 11 in Appendix B.2 report qualitative evaluation of predictive uncertainty on the MNIST dataset.

3.5 Uncertainty evaluation: test examples from known vs unknown classes

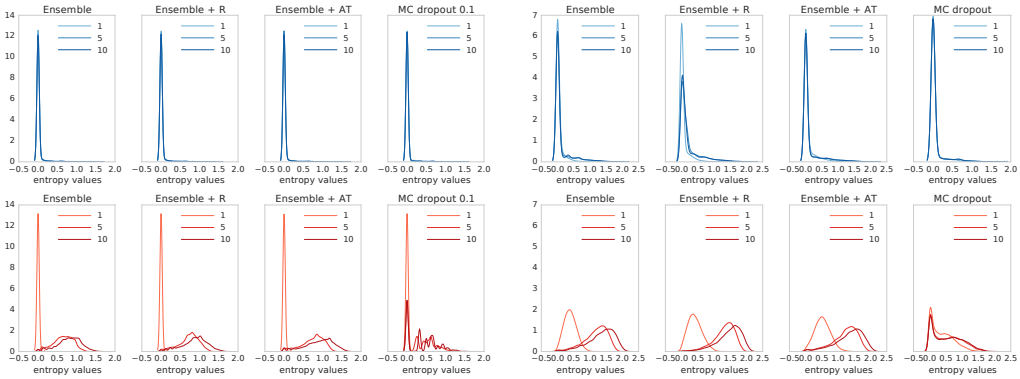
In the final experiment, we evaluate uncertainty on out-of-distribution examples from unseen classes. Overconfident predictions on unseen classes pose a challenge for reliable deployment of deep learning models in real world applications. We would like the predictions to exhibit higher uncertainty when the test data is very different from the training data. To test if the proposed method possesses this desirable property, we train a MLP on the standard MNIST train/test split using the same architecture as before. However, in addition to the regular test set with known classes, we also evaluate it on a test set containing unknown classes. We used the test split of the NotMNIST[10] dataset. The images in this dataset have the same size as MNIST, however the labels are alphabets instead of digits. We do not have access to the true conditional probabilities, but we expect the predictions to be closer to uniform on unseen classes compared to the known classes where the predictive probabilities should concentrate on the true targets. We evaluate the entropy of the predictive distribution and use this to evaluate the quality of the uncertainty estimates. The results are shown in Figure 3(a). For known classes (top row), both our method and MC-dropout have low entropy as expected. For unknown classes (bottom row), as M increases, the entropy of deep ensembles increases much faster than MC-dropout indicating that our method is better suited for handling unseen test examples. In particular, MC-dropout seems to give high confidence predictions for some of the test examples, as evidenced by the mode around 0 even for unseen classes. Such overconfident wrong predictions can be problematic in practice when tested on a mixture of known and unknown classes, as we will see in Section 3.6. Comparing different variants of our method, the mode for adversarial training increases slightly faster than the mode for vanilla ensembles indicating that adversarial training is beneficial

[8] The architecture is similar to the one described in http://torch.ch/blog/2015/07/30/cifar.html.

[9] More precisely, we define disagreement as $\sum_{m=1}^{M} \mathrm{KL}(p_{\theta_m}(y|\mathbf{x})||p_E(y|\mathbf{x}))$ where KL denotes the Kullback-Leibler divergence and $p_E(y|\mathbf{x}) = M^{-1} \sum_m p_{\theta_m}(y|\mathbf{x})$ is the prediction of the ensemble.

[10] Available at http://yaroslavvb.blogspot.co.uk/2011/09/notmnist-dataset.html

for quantifying uncertainty on unseen classes. We qualitatively evaluate results in Figures 12(a) and 12(b) in Appendix B.2. Figure 12(a) shows that the ensemble agreement is highest for letter '*I*' which resembles 1 in the MNIST training dataset, and that the ensemble disagreement is higher for examples visually different from the MNIST training dataset.



(a) MNIST-NotMNIST (b) SVHN-CIFAR10

Figure 3: : Histogram of the predictive entropy on test examples from known classes (top row) and unknown classes (bottom row), as we vary ensemble size M.

We ran a similar experiment, training on SVHN and testing on CIFAR-10 [31] test set; both datasets contain $32 \times 32 \times 3$ images, however SVHN contains images of digits whereas CIFAR-10 contains images of object categories. The results are shown in Figure 3(b). As in the MNIST-NotMNIST experiment, we observe that MC-dropout produces over-confident predictions on unseen examples, whereas our method produces higher uncertainty on unseen classes.

Finally, we test on ImageNet by splitting the training set by categories. We split the dataset into images of dogs (known classes) and non-dogs (unknown classes), following Vinyals et al. [58] who proposed this setup for a different task. Figure 5 shows the histogram of the predictive entropy as well as the maximum predicted probability (i.e. confidence in the predicted class). We observe that the predictive uncertainty improves on unseen classes, as the ensemble size increases.

3.6 Accuracy as a function of confidence

In practical applications, it is highly desirable for a system to avoid overconfident, incorrect predictions and fail gracefully. To evaluate the usefulness of predictive uncertainty for decision making, we consider a task where the model is evaluated only on cases where the model's confidence is above an user-specified threshold. If the confidence estimates are well-calibrated, one can trust the model's predictions when the reported confidence is high and resort to a different solution (e.g. use human in a loop, or use prediction from a simpler model) when the model is not confident.

We re-use the results from the experiment in the previous section where we trained a network on MNIST and test it on a mix of test examples from MNIST (known classes) and NotMNIST (unknown

M	Top-1 error %	Top-5 error %	NLL	Brier Score $\times 10^{-3}$
1	22.166	6.129	0.959	0.317
2	20.462	5.274	0.867	0.294
3	19.709	4.955	0.836	0.286
4	19.334	4.723	0.818	0.282
5	19.104	4.637	0.809	0.280
6	18.986	4.532	0.803	0.278
7	18.860	4.485	0.797	0.277
8	18.771	4.430	0.794	0.276
9	18.728	4.373	0.791	0.276
10	18.675	4.364	0.789	0.275

Figure 4: Results on ImageNet: Deep Ensembles lead to lower classification error as well as better predictive uncertainty as evidenced by lower NLL and Brier score.

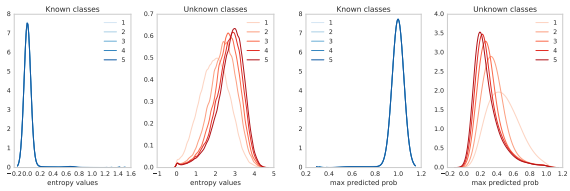


Figure 5: ImageNet trained only on dogs: Histogram of the predictive entropy (left) and maximum predicted probability (right) on test examples from known classes (dogs) and unknown classes (non-dogs), as we vary the ensemble size.

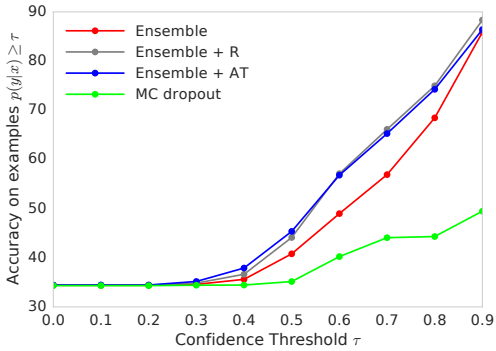


Figure 6: Accuracy vs Confidence curves: Networks trained on MNIST and tested on both MNIST test containing known classes and the NotMNIST dataset containing unseen classes. MC-dropout can produce overconfident wrong predictions, whereas deep ensembles are significantly more robust.

classes). The network will produce incorrect predictions on out-of-distribution examples, however we would like these predictions to have low confidence. Given the prediction $p(y = k|\mathbf{x})$, we define the predicted label as $\hat{y} = \arg\max_k p(y = k|\mathbf{x})$, and the confidence as $p(y = \hat{y}|\mathbf{x}) = \max_k p(y = k|\mathbf{x})$. We filter out test examples, corresponding to a particular confidence threshold $0 \leq \tau \leq 1$ and plot the accuracy for this threshold. The confidence vs accuracy results are shown in Figure 6. If we look at cases only where the confidence is $\geq 90\%$, we expect higher accuracy than cases where confidence $\geq 80\%$, hence the curve should be monotonically increasing. If the application demands an accuracy x%, we can trust the model only in cases where the confidence is greater than the corresponding threshold. Hence, we can compare accuracy of the models for a desired confidence threshold of the application. MC-dropout can produce overconfident wrong predictions as evidenced by low accuracy even for high values of τ, whereas deep ensembles are significantly more robust.

4 Discussion

We have proposed a simple and scalable non-Bayesian solution that provides a very strong baseline on evaluation metrics for predictive uncertainty quantification. Intuitively, our method captures two sources of uncertainty. Training a probabilistic NN $p_\theta(y|\mathbf{x})$ using proper scoring rules as training objectives captures ambiguity in targets y for a given $\mathbf{x}$. In addition, our method uses a combination of ensembles (which captures "model uncertainty" by averaging predictions over multiple models consistent with the training data), and adversarial training (which encourages local smoothness), for robustness to model misspecification and out-of-distribution examples. Ensembles, even for $M = 5$, significantly improve uncertainty quality in all the cases. Adversarial training helps on some datasets for some metrics and is not strictly necessary in all cases. Our method requires very little hyperparameter tuning and is well suited for large scale distributed computation and can be readily implemented for a wide variety of architectures such as MLPs, CNNs, etc including those which do not use dropout e.g. residual networks [22]. It is perhaps surprising to the Bayesian deep learning community that a non-Bayesian (yet probabilistic) approach can perform as well as Bayesian NNs. We hope that our work will encourage the community to consider non-Bayesian approaches (such as ensembles) and other interesting evaluation metrics for predictive uncertainty. Concurrent with our work, Hendrycks and Gimpel [23] and Guo et al. [20] have also independently shown that non-Bayesian solutions can produce good predictive uncertainty estimates on some tasks. Abbasi and Gagné [1], Tramèr et al. [57] have also explored ensemble-based solutions to tackle adversarial examples, a particularly hard case of out-of-distribution examples.

There are several avenues for future work. We focused on training independent networks as training can be trivially parallelized. Explicitly de-correlating networks' predictions, e.g. as in [37], might promote ensemble diversity and improve performance even further. Optimizing the ensemble weights, as in stacking [60] or adaptive mixture of experts [28], can further improve the performance. The ensemble has M times more parameters than a single network; for memory-constrained applications, the ensemble can be distilled into a simpler model [10, 26]. It would be also interesting to investigate so-called *implicit ensembles* the where ensemble members share parameters, e.g. using multiple heads [36, 48], snapshot ensembles [27] or swapout [52].